27. Dynamic Programming Patterns

27. Dynamic Programming Patterns

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2 days
Slug: `27-dynamic-programming`

Learning objectives

- Identify overlapping subproblems
- Write bottom-up DP tables
- Memoize top-down recursion
- Apply classic patterns (knapsack, LIS, paths)

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Idea

DP stores answers to subproblems to avoid recomputation.

```
// Climb stairs: ways(n) = ways(n-1)+ways(n-2)
func Climb(n int) int {
    if n < 2 { return 1 }
    a, b := 1, 1
    for i := 2; i <= n; i++ {
        a, b = b, a+b
    }
    return b
}
```

Pattern checklist

1. Define state clearly (`dp[i]` means …)
2. Write recurrence
3. Determine base cases
4. Choose iteration order
5. Optimize space if only last states needed

Infra relevance

DP appears in scheduling, rate allocation, tokenization costs, and alignment-like problems — not only interview questions.

Common mistakes (especially from Python instincts)

- Memoizing without clear state definition
- Off-by-one in table sizes

- Exponential backtracking labeled as DP

Exercises

Work in `exercises/27-dynamic-programming/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/27-dynamic-programming/` until you have an honest failing attempt.

Self-check

1. Overlapping subproblems meaning?
2. Top-down vs bottom-up?
3. When reduce DP space?

Further reading

- https://en.wikipedia.org/wiki/Dynamic_programming